A Simulated Flash-Sale Distributed Data Pipeline: Integrating Kafka, Spark, and Citus on Docker Compose

Mete Kılıç and İsmail Emre Yıldız
Student IDs: 231401028, 221401006
Department of Computer / Artificial Intelligence Engineering
TOBB University of Economics and Technology, Ankara, Türkiye
BIL 401 — Big Data and Distributed Data Processing (Systems Track), 2025–2026 Summer Term

Abstract*—Flash sales—short, high-intensity sales windows in which traffic can spike by one to two orders of magnitude within seconds—stress backend systems both through raw volume and through the automated scalper traffic they attract. This paper presents a self-contained simulation of that scenario built from three distinct open-source distributed technologies. A synthetic clickstream generator feeds an Apache Kafka cluster; Apache Spark Structured Streaming consumes the stream and separates apparent bot traffic from legitimate traffic in near real time; and the results are persisted in Citus, a horizontally-sharded PostgreSQL cluster, with Prometheus and Grafana providing live observability. The complete stack—seventeen containers—runs on a single development laptop under Docker Compose. Rather than merely asserting the properties of interest, we injected faults and scaled components deliberately: Kafka brokers were stopped one and two at a time, a Citus worker was killed mid-write, the Spark cluster was scaled from one to three workers, a third Citus worker was added and its shards rebalanced, and the pipeline was driven at five times its nominal target load. Every reported figure derives from a logged run on the system. The pipeline tolerates the failures it was configured to tolerate and fails explicitly once those limits are exceeded; scaling Spark and Citus each produced a measurable, repeatable improvement; and the bottleneck under extreme load proved to be the single-process data generator rather than any distributed component. We further identified, and subsequently corrected, a systematic weakness in the anomaly-detection rule under backlog conditions, verifying the corrected pipeline against the generator's known ground-truth bot ratio.

Index Terms*—Distributed systems, Apache Kafka, Apache Spark, Structured Streaming, Citus, PostgreSQL, fault tolerance, horizontal scalability, stream processing, anomaly detection, Docker Compose.

I. Introduction

E-commerce flash sales create two related but distinct engineering problems. The first concerns scale: a system that comfortably handles a few hundred requests per second must survive a burst that is fifty to one hundred times larger for a short but unpredictable window. The second concerns abuse: those same burst conditions are precisely when scalper bots are most active, because the incentive to acquire limited-stock items before other buyers is highest. A system that scales but cannot distinguish bots from customers does not solve the business problem even if it remains available.

For the Systems Track, the assigned brief was to integrate at least three or four distinct open-source distributed technologies, deploy them with Docker Compose so as to simulate a distributed environment on a single laptop, and then to genuinely test—not merely assert—scalability, connectivity, and fault tolerance. We selected a deliberately simplified flash-sale pipeline as the vehicle for this exercise, because it naturally requires an ingestion buffer (Kafka), a stream processor capable of applying business logic in near real time (Spark), and a store able to absorb high-volume, shardable writes (Citus)—three technologies with genuinely different roles rather than three interchangeable databases.

The technology combination was guided by the project brief, which lists "distributed PostgreSQL scalability tests – Citus" and "Spark–Kafka connection, scaling" as separate suggested topics; integrating both into a single pipeline provided a more demanding exercise than either in isolation. In hindsight this proved to be the harder path: three technologies that must agree on hostnames, ports, and restart timing produce genuine integration defects that a single-technology demonstration does not, and Section VIII records several of them.

This paper is organized as follows. Section II reviews the related work that shaped several design choices. Section III describes the deployed architecture, and Section IV the implementation details worth calling out specifically. Section V states the experimental setup. Section VI presents the fault-tolerance, scalability, and stress-test results. Section VII discusses the limitations encountered, including ones we did not anticipate, and Section VIII the practical lessons learned. Section IX concludes.

II. Related Work

The overall pattern—a log-structured broker buffering writes for a stream processor, which in turn writes into a horizontally-scaled store—is long-standing in industry, tracing back to Kafka's original role as a

distributed commit log for decoupling producers and consumers [4]. Spark Structured Streaming's micro-batch execution model [2] is among the more common ways to bridge declarative streaming with batch-style custom logic; we specifically required its `foreachBatch` sink rather than the purely declarative API, because per-batch IP counting and upsert-style aggregate writes are not naturally expressible as a single continuous streaming query.

Citus turns a regular PostgreSQL cluster into a sharded, horizontally-scalable database while preserving standard SQL, and is used in production for high-write, time-series-like workloads [3], [6]: a coordinator node routes and plans queries, while a configurable number of worker nodes each hold a slice (shard) of every distributed table. We chose Citus because—unlike, for example, adding read replicas—it permits a demonstration of genuine horizontal write scaling and shard rebalancing, which is closer in spirit to the "distributed PostgreSQL scalability tests" suggested in the brief.

On the anomaly-detection side, our rule—flagging any IP address responsible for more than a threshold number of events within a single processing window—is a deliberately simplified form of the rate-limiting and bot-scoring techniques discussed around ticket and sneaker resale bots, where production systems typically combine rate limiting with CAPTCHAs, device fingerprinting, and behavioral scoring rather than a single fixed threshold. We adopted the simplest version intentionally, since the focus of the project is the distributed-systems engineering around the rule rather than the rule itself. Section VII discusses why this simplicity mattered more than expected once the system was placed under sustained heavy load.

III. System Architecture

Fig. 1 shows the deployed architecture. All components communicate over a single Docker bridge network and are defined in one docker-compose.yml file comprising seventeen services.

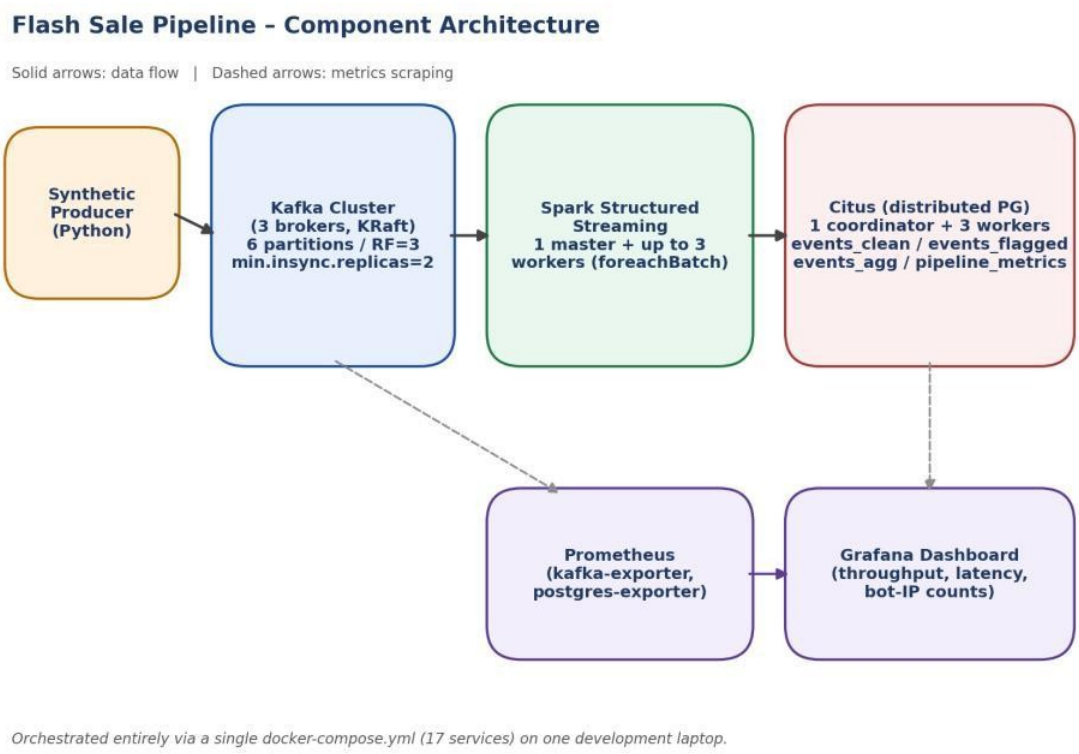

Fig. 1. Component architecture of the flash-sale pipeline. Solid arrows denote the data path; dashed arrows denote metrics scraping for the monitoring stack. Orchestrated via a single docker-compose.yml (17 services) on one development laptop.

A. Synthetic Data Generator

A single Python process (using the `kafka-python` client) generates a synthetic e-commerce clickstream; each event carries a user identifier, an action (VIEW / ADD_TO_CART / PURCHASE, realistically weighted), a product identifier, a timestamp, and an IP

address. The generator automatically cycles between a NORMAL phase (≈100 events/s, no bot traffic) and a FLASH_SALE phase (≈10,000 events/s target, with roughly 15% of traffic produced by a small, fixed pool of bot identities targeting a handful of "hot" products). A manual-trigger mode (a watched control file) is also supported so that bursts can be fired on demand during a live demonstration.

B. Kafka Layer

Three Kafka brokers run in KRaft mode (no ZooKeeper), each acting as both broker and controller. The clickstream-events topic has six partitions, replication factor three, and `min.insync.replicas=2`. This is the load-bearing configuration behind most of the Kafka fault-tolerance results in Section VI: it is intended to tolerate the loss of exactly one broker of three, but to refuse writes—rather than silently lose them—if a second broker is lost simultaneously. Verifying that this is what actually happens, and not merely what the documentation states, is treated as one of the more important results of this work.

C. Stream Processing (Spark Structured Streaming)

Spark reads the topic with `startingOffsets=latest` and a one-minute watermark, then uses a `foreachBatch` sink so that each micro-batch (one trigger interval, five seconds by default) is delivered as a regular, non-streaming DataFrame. For each batch the pipeline: (1) counts events per IP address and flags any IP above a configured threshold as bot-like; (2) splits the batch into legitimate and flagged sets and appends both, with their reason, into Citus; (3) aggregates the legitimate set by product and action and upserts the running totals into a small aggregate table; and (4) writes one row of batch-level metrics (size, latency, flagged count) for monitoring. The Spark cluster comprises one master plus a configurable number of workers; we evaluated one and three workers (Section VI-D).

D. Storage Layer (Citus)

Citus stores the pipeline output as distributed tables across one coordinator and (after a mid-project scale-out; Section VI-E) three worker nodes: `events_clean` (sharded by `product_id`), `events_flagged` (sharded by `ip_address`, with a reason column), and `events_agg` (sharded by `product_id`, holding running per-product/action counts). A small `pipeline_metrics` table is kept as a regular coordinator-local table, since it is low-volume and used only by the dashboard.

E. Monitoring

Prometheus scrapes a `kafka-exporter` and a `postgres-exporter`; Grafana reads `pipeline_metrics` directly from Citus and the Kafka offset-rate metric from Prometheus, on a single dashboard (Fig. 2) covering total, legitimate, and flagged event counts, micro-batch latency, distinct flagged-IP counts, top products, and Kafka throughput.

F. Rationale for Docker Compose over Kubernetes

Kubernetes (via minikube or kind) was considered at the planning stage, as the more common answer to running distributed systems. It was rejected for two concrete reasons. First, the brief explicitly requires something runnable and testable on a student laptop rather than a cluster. Second, adding a second orchestration layer atop an already seventeen-service Compose file would have required writing and debugging Kubernetes manifests for the same services without contributing anything to the systems question under study—the integration, scaling, and fault tolerance of Kafka, Spark, and Citus. Docker Compose already provides independently restartable, independently scalable containers on a private network, which is precisely what "simulate a distributed environment locally" calls for.

IV. Implementation Details

A. Anomaly-Detection Rule

In its final, submitted form, the rule flags any IP address whose events-per-second rate exceeds `ANOMALY_RATE_PER_SEC_THRESHOLD` (5.0 by default), where the rate is computed against the batch's own actual time span (maximum minus minimum event timestamp), not the nominal five-second trigger interval. This was not the original design: the first version used a fixed per-batch event count (25 events per batch) with no time normalization, which is simpler to reason about but breaks down under backlog conditions, as Section VII-A describes in detail. We retain that account as a narrative of how the final rule was reached and describe only the corrected rule here.

B. Sharding Strategy

`events_clean` and `events_agg` are sharded by `product_id`, which keeps all activity for a given product on one shard—useful for the top-products queries run for the dashboard. `events_flagged` is sharded by `ip_address` instead, since the natural follow-up query for that table is "what did this IP do," not "what happened to this product." `pipeline_metrics` is not distributed; at roughly one row every five seconds it is far too small to warrant sharding, and keeping it local to the coordinator simplifies the Grafana queries.

C. Synthetic Data Design

Legitimate traffic is drawn from a large pool of distinct users and IP addresses (40,320 IPs in the final configuration; Section VII-A explains why this number matters and how it was reached) with realistic action weighting (mostly browsing, occasional add-to-cart, fewer purchases). Bot traffic is drawn from a small, fixed pool of eight identities and IPs and is weighted toward PURCHASE, modeling scripts that exist to acquire stock the instant it becomes available rather than to browse. This asymmetry—many legitimate identities, few repeated bot identities—is what makes the per-IP rate threshold workable under normal conditions.

V. Experimental Setup

All results were collected on a single Windows 11 laptop running Docker Desktop with the WSL2 backend (Ubuntu distribution), with sixteen logical CPUs and roughly 12–15 GB of RAM available to WSL2 at the time of testing. No cloud resources were used, in line with the brief's requirement that the system be runnable and testable on a student laptop. Table I summarizes the component versions and configuration.

TABLE I. Component Versions and Configuration

Component	Image / Version	Configuration
Kafka	apache/kafka:3.7.1	3 brokers, KRaft, 6 partitions, RF=3, min.insync.replicas=2
Spark	Apache Spark 3.5.1 (custom image)	1 master, 1–3 workers, 2 cores / 2 GB per worker
Citus	citusdata/citus:12.1	1 coordinator, 2–3 workers
Monitoring	Prometheus + Grafana (latest)	kafka-exporter, postgres-exporter
Orchestration	Docker Compose	17 services, single bridge network

Every test was run against a continuously-operating pipeline (the producer's automatic NORMAL/FLASH_SALE cycle ran for the system's entire multi-day lifetime, except where a manual stress test temporarily overrode it), so the "before" state for every fault-injection or scaling test reflects a system already holding millions of events, not a freshly-booted, empty one.

VI. Results

A. End-to-End Correctness

Fig. 2 shows the Grafana dashboard during a normal operating window that includes one flash-sale burst. The total/legitimate/flagged panel renders the burst as a clear spike, the latency panel shows processing time rising during the burst and recovering afterward, and the distinct-flagged-IPs panel shows the detection rule catching a sharp increase in the small number of IPs seeded as scalper traffic.

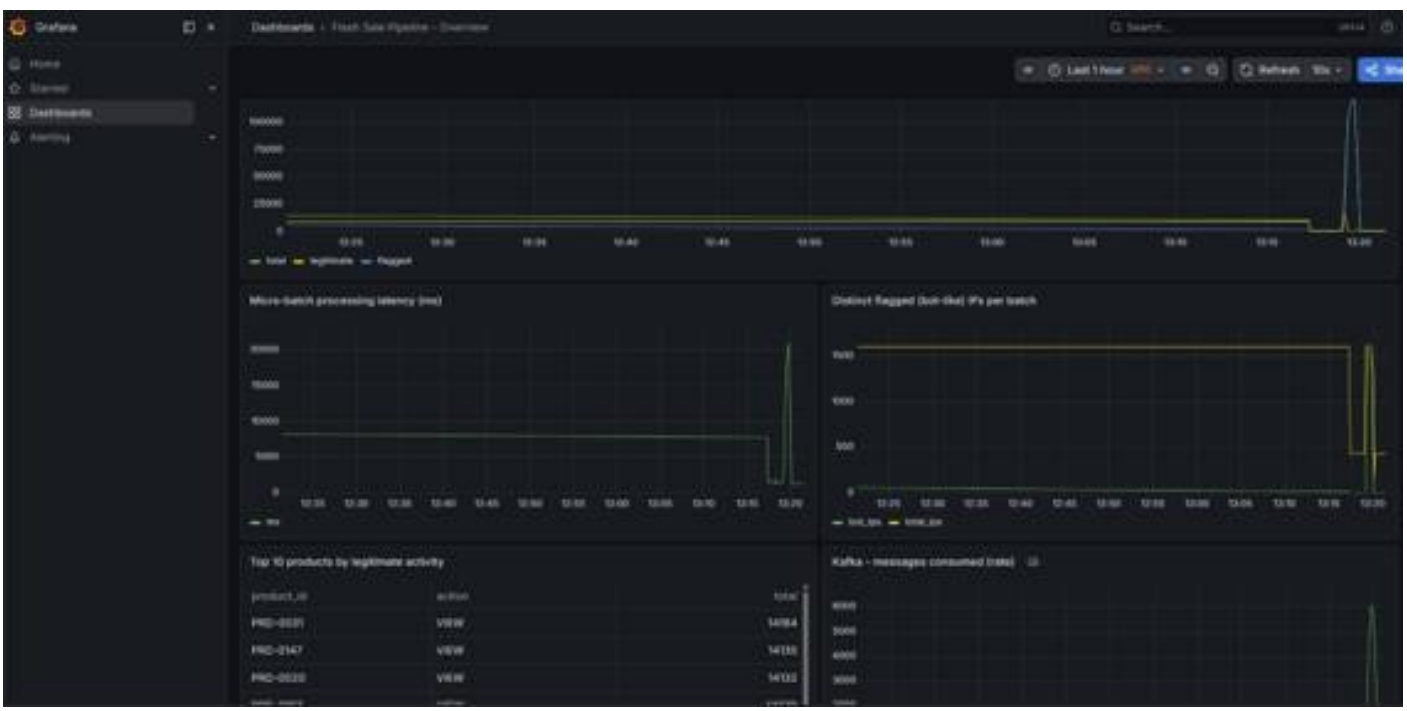

Fig. 2. Grafana dashboard during a flash-sale burst: total/legitimate/flagged events, micro-batch latency, distinct flagged IPs, top products, and Kafka throughput, all read live from the pipeline.

B. Fault Tolerance – Kafka

Two scenarios were tested: losing one broker of three (within the configured tolerance) and losing two of three (deliberately exceeding it).

1) Single broker loss (within tolerance): Stopping kafka-3 while the producer continued sending data had no visible effect on the pipeline: `pipeline_metrics` continued to receive new rows at the normal cadence, with a brief processing-time bump (one batch took 22.3 s instead of the usual 1–2 s) rather than any gap or error. Restarting the broker returned it to the controller quorum within roughly 15 s, with no manual intervention.

2) Double broker loss (beyond tolerance, on purpose): Stopping kafka-2 and kafka-3 simultaneously—leaving only kafka-1, with two of three in-sync replicas unavailable—caused the producer to immediately begin failing with `NotEnoughReplicasError` on every partition, exactly as the `min.insync.replicas=2` setting is meant to guarantee. Spark's consumer simultaneously logged a DNS resolution failure reaching kafka-2. Crucially, the system did not silently drop events; it refused new writes and reported so repeatedly in the producer logs. Restarting kafka-2 (restoring two of three brokers) allowed automatic recovery once the health check passed; producer errors stopped and `pipeline_metrics` resumed advancing within about a minute, without restarting the producer or the Spark job.

TABLE II. Kafka Fault-Injection Outcomes

Scenario	Brokers lost	Effect	Recovery
Single broker loss	1 of 3	No visible impact; one elevated-latency batch (≈22 s)	Automatic, ≈15 s
Double broker loss	2 of 3	Producer correctly rejected (NotEnoughReplicasError); no silent data loss	Automatic once 2/3 healthy, ≈60–90 s

C. Fault Tolerance – Citus

Stopping one of the two Citus workers mid-write produced an immediate, explicit PostgreSQL error on the Spark side ("connection to the remote node … failed: could not translate host name"), surfaced through several batch-level JDBC exceptions ("Batch entry … was aborted"). In one run this crashed the Spark streaming application entirely; Docker's `restart: on-failure` policy brought it back automatically (observed RestartCount 2 on the stream-processor container), and because Spark advances its Kafka checkpoint only after a batch fully succeeds, no data was permanently lost—the failed batch's events remained in Kafka and were reprocessed after the restart. Across the incident, `pipeline_metrics` shows no permanent gap in batch identifiers, only a temporary slowdown.

This is, by design, not the same guarantee Kafka provides. Citus is not run with shard replication (`citus.shard_replication_factor=1`, for simplicity), so a downed worker genuinely cannot serve the shards it holds until it returns—there is no automatic shard failover. We regard this as an honest and useful result rather than a flaw to conceal: it demonstrates the practical difference between a system configured for node-loss tolerance (Kafka) and one that is not (Citus, as configured here) using the same fault-injection test against both.

D. Scalability – Spark

Micro-batch processing time was compared at one worker versus three workers on naturally-occurring flash-sale batches of comparable size (identical batch sizes were not forced; the closest matches occurring during the automatic burst cycle were captured).

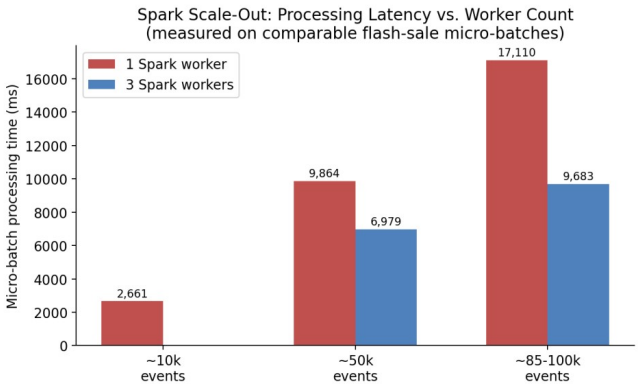

Fig. 3. Micro-batch processing latency at 1 vs. 3 Spark workers on comparable flash-sale batch sizes.

At roughly 50,000 events per batch, three workers cut processing time from 9,864 ms to 6,979 ms (about 29% faster). At roughly 85,000–100,000 events, the gap widened to 17,110 ms versus 9,683 ms (about 43% faster, close to a 1.8× speed-up). The single-worker run also accumulated a noticeably larger backlog under sustained burst load (one batch reached 140,560 events) than the three-worker run (largest observed batch 96,488 events); scaling out therefore not only made individual batches faster but also kept the system from falling as far behind.

Scaling tests were deliberately stopped at three workers. The Kafka topic has six partitions, and Spark's read-side parallelism from a Kafka source is capped at one task per partition. Three workers at two cores each provide six cores, matching the partition count; beyond that point additional workers would not increase Kafka-

read parallelism and would help only downstream aggregation and JDBC-write stages, which are not the bottleneck at the loads tested. Three workers is thus a reasoned stopping point given the topic configuration, not an arbitrary one.

E. Scalability – Citus

A third Citus worker was added mid-project to an already-populated cluster and Citus's built-in `rebalance_table_shards()` function was run (in `block_writes` transfer mode, since the append-only tables lack a primary key / replica identity, which the default logical-replication transfer mode requires).

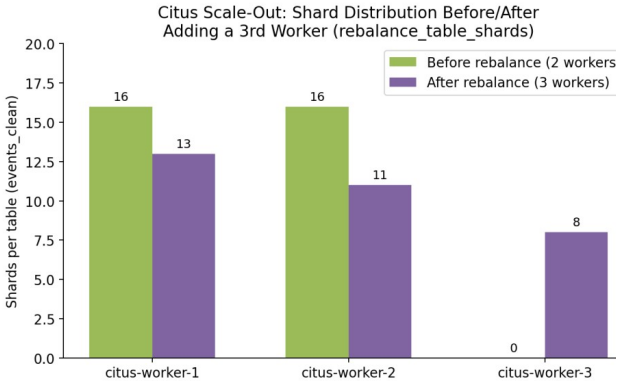

Fig. 4. Shard distribution for events_clean before and after adding a third Citus worker and rebalancing (identical pattern observed for events_flagged and events_agg).

Before the rebalance, all 32 shards of each distributed table sat on the original two workers (16/16). After rebalancing, the distribution became 13/11/8 across the three workers—not perfectly even, but Citus's rebalancer optimizes for overall data-movement cost rather than a perfectly equal split, and a few extra shards on the older workers is an acceptable trade-off. Row counts on all three tables immediately before and after the operation (events_clean: 14,556,299; events_flagged: 76,327,662; events_agg: 2,328,579) were verified identical, confirming that the rebalance moved data without loss or duplication while the pipeline kept writing throughout.

F. Stress Test – 5× Overload

To locate the point at which the system would actually break, the producer was temporarily reconfigured for a manually-triggered burst at 50,000 events/s—five times the normal flash-sale target—against the three-worker Spark / three-worker Citus configuration.

The first observation was that the producer itself could not reach 50,000 events/s: a 30-second target phase took 138 s of wall-clock time to send its intended 1,500,000 events, which works out to a sustained throughput of about 10,870 events/s regardless of the requested figure. The single-process Python generator—not Kafka, Spark, or Citus—was therefore the actual ceiling on applied load, and that ceiling sits almost exactly at the "normal" flash-sale rate already in use. A substantial part of the intended extreme-load test thus measured the test tool rather than the system under test.

The consumption side did not idle during those 138 s, nor did it crash. Batch sizes climbed steadily and then leveled off in the 150,000–160,000 event range, with processing time stabilizing around 14,000–14,700 ms per batch; the system reached a steady state rather than falling over or growing its backlog without bound. That steady-state throughput (≈150k events / 14.5 s ≈ 10,300–11,000 events/s) closely matches the producer's measured ceiling, suggesting that at this hardware scale producer and consumer throughput happen to be well matched. Which side would give first if the producer were parallelized cannot yet be determined; it is listed as future work (Section VII-C).

The most useful result of this test was not a number but a defect: once batches grew large enough, the anomaly threshold began flagging nearly all traffic as bot-like, including ordinary users, simply because a large micro-batch naturally accumulates more events per IP than a fixed per-batch count assumes. Totals were observed in which legitimate events dropped to zero and effectively the entire batch was marked flagged. This was not a one-off glitch: the same pattern appeared independently in three separate runs (the one-worker scaling baseline, the three-worker comparison, and this stress test). Section VII-A treats this as a design limitation rather than an implementation bug.

G. Connectivity Testing

The brief asks specifically for connectivity testing alongside scalability and fault tolerance, so we report it as its own result. "Connectivity" is treated as whether every component can actually find and authenticate to every other component it depends on—not merely that each container starts and stays up.

- *Kafka:* `kafka-topics.sh --describe` was run after every broker restart and consistently showed all six partitions with a full three-broker in-sync replica set whenever all brokers were healthy.
- *Citus:* the coordinator's `pg_dist_node` / `v_cluster_nodes` view was queried after every container restart and scale-out, and was required to show every worker as `isactive = t` before any subsequent test result was trusted.
- *Spark:* the master UI worker list (Fig. 5) was checked before every scaling test to confirm the

expected number of workers had registered, rather than assuming that `docker compose up` had silently succeeded.

- *End-to-end:* the Spark streaming job is itself a continuous connectivity test—the moment it cannot reach either Kafka or Citus, this appears immediately as exceptions in its logs. The 7,764 micro-batches summarized in the Appendix therefore also represent 7,764 passed end-to-end connectivity checks across the whole stack.

VII. Discussion and Limitations

A. The Anomaly Threshold Was Not Rate-Aware

The original IP-flagging rule counted raw events per IP within whatever the current micro-batch happened to contain, with no normalization for the wall-clock time that batch covered. Under normal conditions the five-second trigger interval is a stable proxy for "events in roughly five seconds," so the rule behaved as intended. Under backpressure, however, Spark can process a much larger batch covering 15–20 s of accumulated backlog in one shot, and an ordinary user can accumulate a few dozen page views across that longer window—enough to cross a fixed threshold.

Section VI-F first observed this in individual runs. Analysis of the full accumulated dataset (Appendix, Table IV) then showed it was not an edge case: across all 7,764 micro-batches processed to that point, 84.00% of roughly 93 million total events had been flagged as bot-like—clearly not a realistic scalper rate. Rather than listing this as a limitation and moving on, we corrected it in two iterations.

The first correction replaced the fixed event-count threshold with a rate-normalized one: instead of "more than 25 events in this batch," the rule became "more than 5 events per second, measured against the batch's actual event-timestamp span." This was an improvement—on one comparable large batch (≈106,700 events) it reduced the flagged share from 100% to 74.6%—but remained far above the 15% ground-truth bot ratio. Examining which IPs were still flagged revealed the second, independent cause: the synthetic "normal" traffic pool contained only 1,600 distinct IP addresses, and a batch of over 100,000 events spread across 1,600 IPs implies roughly 67 events per IP on average—enough to look statistically anomalous however the time window is normalized. The rate-normalization fix had addressed a real defect, but it sat atop a second one in the test-data design.

The legitimate-IP pool was widened to 40,320 addresses, the producer image rebuilt, and the same flash-sale scenario re-run. The result, measured directly against event timestamps rather than batch identifiers (to remove any ambiguity from container restarts): in a clean 18-second flash-sale window, exactly 8 distinct IP addresses were flagged—all 8 the seeded bot pool (198.51.100.1–8), with no ordinary IP among them—at a 15.0% flagged share, matching the generator's seeded ratio of 0.15 almost exactly. Fig. 6 shows all three stages side by side.

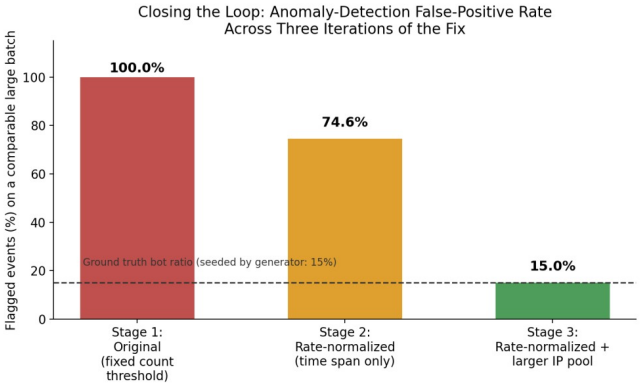

Fig. 6. False-positive rate on comparable large flash-sale batches across the three iterations of the anomaly-detection fix, against the generator's known ground-truth bot ratio (15%, dashed line).

This two-step fix-and-re-diagnose process is worth describing in detail precisely because the first fix looked successful in isolation (a 25-point drop is a real improvement) and it would have been easy to stop there and report the issue as partially mitigated. Only by re-checking against ground truth—the bot ratio we ourselves had seeded—did the incompleteness become visible and the second cause emerge. The corrected logic and the widened IP pool are both included in the submitted code; the verification queries and their output are recorded in the Appendix.

B. Single Points of Failure Accepted on Purpose

- *Citus shard replication factor is 1:* a deliberate simplicity choice, but it means a lost worker's shards are unavailable until the worker returns, with no automatic failover. A production deployment would set `citus.shard_replication_factor ≥ 2` or rely on a replicated storage layer.
- *The synthetic producer is a single Python process:* convenient for a course project, but both a throughput ceiling (Section VI-F) and a single point of failure for data generation. A realistic ingestion tier would use multiple independent producer processes.
- *The Spark job provides at-least-once, not exactly-once, semantics:* because Kafka offsets

advance only after a batch's writes succeed, a crash-and-restart can cause a batch to be reprocessed. This is safe for the append-style writes and idempotent upserts used here, but would require explicit deduplication for some other downstream tables.

C. Future Work

- Replace the single global threshold with a per-IP baseline (flagging relative to that IP's own recent history), which would also catch slower, more patient scraping bots that stay just under a global threshold.
- Parallelize the synthetic producer (multiple processes or a faster client such as `confluent-kafka`) to determine whether Kafka/Spark/Citus have headroom above the ≈11,000 events/s that could be applied here.
- Re-run the Citus worker-loss test with `shard_replication_factor=2` to quantify the cost/benefit of automatic shard failover.

VIII. Practical Lessons Learned

Three lessons shaped the final architecture as much as any deliberate design decision.

- The original plan used Bitnami's prebuilt Kafka and Spark images, as most tutorials recommend. Partway through the project, Broadcom (Bitnami's current owner) removed almost all free, versioned image tags from Docker Hub, breaking the setup on a clean pull. We migrated Kafka to the official `apache/kafka` image and built a custom Spark image from the official Apache binary distribution—a real time cost, but one that reduced dependence on third-party packaging decisions.
- Citus's inter-node authentication is separate from client-facing authentication and would intermittently break after a full stack restart with a "no password supplied" error—including once mid-rebalance. The fix (registering credentials explicitly in Citus's `pg_dist_authinfo` metadata table on every node) is now baked into the startup scripts.
- Grafana's PostgreSQL datasource plugin produced a persistent "no default database configured" error that several rounds of correcting the obvious YAML field did not resolve; it ultimately required clearing Grafana's data volume and re-provisioning from a clean state, after which it has been stable.

IX. Conclusion

We set out to integrate Kafka, Spark, and Citus into a single Docker-Compose-orchestrated pipeline and to genuinely test—not merely assert—its scalability, connectivity, and fault tolerance, using a simulated flash sale as the concrete use case. By the end of the project, all three technologies had documented, repeatable evidence of both fault tolerance and horizontal scalability, gathered from a system that ran continuously for several days and processed tens of millions of synthetic events, rather than from a single clean demonstration run.

Two results were not specifically planned for: a vendor packaging change that forced an unplanned but ultimately beneficial redesign of two of the four core images, and a genuine, repeatable weakness in the anomaly-detection logic that became fully visible only once the accumulated output was treated as a dataset and queried. For the second, we did not stop at reporting it: the behavior was traced to two separate, compounding causes, both corrected, and the fix re-verified against the generator's seeded ground truth (Section VII-A). Both experiences are representative of what systems engineering looks like in practice—plans changing because of dependencies outside one's control, and simple rules behaving correctly right up until the precise conditions where they do not.

A more literal test of resilience occurred near the end of the project: a power outage took down the development machine mid-session, and Docker Desktop's container/image metadata did not survive the unclean shutdown. The underlying Docker volumes, however, did; after rebuilding the four custom images (roughly 15 minutes, mostly re-downloading packages), the system came back up with all previously-processed events intact in Citus (by that point roughly 124 million, up from the ≈93 million reported in the Appendix) and resumed normal operation without manual data recovery. Unplanned though it was, this demonstrated exactly the property Section VI set out to verify: the stack's state lives in its volumes, not in any single running process.

In summary: every individual component (Kafka, Spark, Citus, and even the producer) is simpler in isolation than at the moment they must all depend on one another simultaneously, and most of what we learned came from the seams between them rather than from any one component alone.

References

[1] Apache Kafka Documentation. [Online]. Available: https://kafka.apache.org/documentation/

[2] M. Armbrust *et al.*, "Structured Streaming: A declarative API for real-time applications in Apache Spark," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2018, pp. 601–613.

[3] Citus Data Documentation. [Online]. Available: https://docs.citusdata.com/

[4] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB*, 2011.

[5] Docker Compose Documentation. [Online]. Available: https://docs.docker.com/compose/

[6] U. Cubukcu *et al.*, "Citus: Distributed PostgreSQL for data-intensive applications," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2021, pp. 2490–2502.

[7] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.

Appendix

A. Repository / Submission Structure

- `docker-compose.yml` – full 17-service orchestration.
- `producer/` – synthetic clickstream generator (Python).
- `spark/jobs/streaming_job.py` – the Spark Structured Streaming application.
- `citus/init/` – coordinator/worker bootstrap SQL.
- `monitoring/` – Prometheus config and Grafana provisioning.
- `scripts/` – fault_injection.sh, healthcheck.sh, trigger_burst.sh, sample_queries.sql.
- `README.md` – setup and run instructions.

Full source code, this report, and the Checkpoint 2 status report are available at: https://github.com/Metekilic0/Flash-sale-system-project.

B. Spark Worker Cluster at Three Workers

Fig. 5 is the Spark master UI captured during the scale-out test of Section VI-D, confirming three alive workers (six cores total) with the FlashSaleStreamProcessor application running.

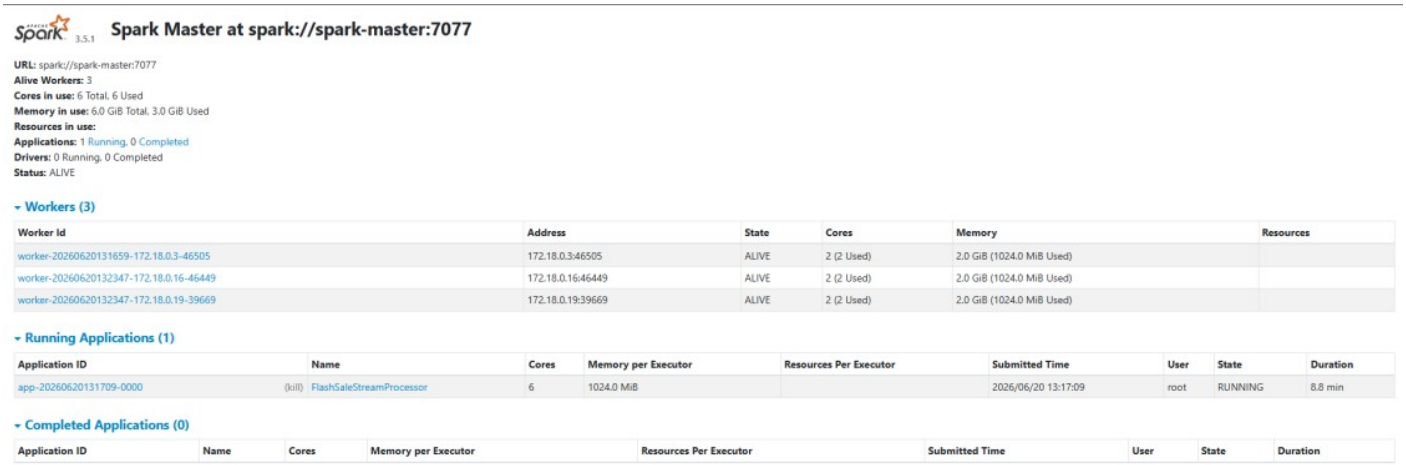

Fig. 5. Spark master UI confirming 3 alive workers (6 cores total) during the scale-out test of Section VI-D, with the FlashSaleStreamProcessor application running.

C. Analytics on the Accumulated Dataset

By the time this report was written, the pipeline had accumulated 7,764 micro-batches and roughly 93 million synthetic events across several days of near-continuous operation. The queries below were run directly against the live Citus cluster (full SQL in scripts/sample_queries.sql).

TABLE III. Top Products by Legitimate View Activity (all-time)

Rank	Product ID	Action	Total Events
1	PRD-0093	VIEW	52,594
2	PRD-0059	VIEW	52,580
3	PRD-0103	VIEW	52,536
4	PRD-0030	VIEW	52,513

Rank	Product ID	Action	Total Events
5	PRD-0022	VIEW	52,488
6	PRD-0048	VIEW	52,460
7	PRD-0194	VIEW	52,447
8	PRD-0082	VIEW	52,430
9	PRD-0110	VIEW	52,425
10	PRD-0060	VIEW	52,421

The top ten products fall within about 0.3% of each other, exactly as expected from legitimate traffic drawn uniformly from a 200-item catalog with no built-in popularity bias. Had the "legitimate" traffic shown artificial concentration, it would have indicated a bug in the generator upstream of anything under test; it did not.

TABLE IV. Overall Legitimate vs. Flagged Split (all-time)

legit_total	flagged_total	flagged_pct
14,870,365	78,082,372	84.00%

This is the number discussed in Section VII-A. It does not mean the system is under bot attack 84% of the time; it means the anomaly rule's assumptions break down specifically during the large, backlogged batches that follow a flash-sale burst or a fault-injection test, and those batches are large enough to dominate the all-time total despite being a minority of all batches.

TABLE V. Top 10 IP Addresses by Flagged Event Count (all-time)

IP Address	Flagged Events	Distinct Batches
198.51.100.6	1,675,550	1,422
198.51.100.7	1,675,538	1,423
198.51.100.3	1,675,217	1,423
198.51.100.4	1,675,200	1,422
198.51.100.8	1,674,974	1,421
198.51.100.2	1,674,446	1,422
198.51.100.5	1,673,742	1,420
198.51.100.1	1,673,442	1,423
10.4.34.14	41,603	941
10.1.51.66	41,568	935

The top eight rows are exactly the seeded bot pool (198.51.100.1–8), each flagged roughly 1.67 million times—expected, and confirming that the rule consistently catches the traffic it was designed to catch. Rows nine and ten (10.4.34.14 and 10.1.51.66) are ordinary addresses from the large legitimate pool, yet each was independently flagged in roughly 935–941 separate micro-batches over the project's lifetime—a recurring pattern across hundreds of independent batches, which is the strongest evidence that the false-positive behavior of Section VII-A is systematic rather than incidental.

TABLE VI. Processing-Latency Distribution and Dataset Scale (all-time)

min_ms	avg_ms	max_ms	batch_count
508	2,517.6	184,072	7,764

The average batch (2.5 s) is comfortably within the five-second trigger interval. The single worst batch (184,072 ms ≈ 184 s) is a genuine outlier and is believed to correspond to the recovery moment immediately following the two-broker Kafka failure test (Section VI-B): roughly two minutes of Kafka data backlogged while only one broker was reachable, arriving at Spark in a single trigger once connectivity was restored. This was not isolated with a dedicated timestamp-matched query, so it is presented as a plausible rather than a proven explanation. Across the whole run, events_clean held 14,882,256 rows, events_flagged 78,150,154, and events_agg 2,378,083, over 7,764 micro-batches—on the order of 93 million synthetic events, none of it from a static file, all generated, streamed, filtered, aggregated, and stored by the system while that same system was being killed, restarted, and scaled out from underneath itself.

D. Reproducing These Results

From a clean checkout, on a machine with Docker Desktop (WSL2 backend on Windows):

- `cp .env.example .env`
- `docker compose up -d --build` (first run pulls/builds all images; subsequent runs are fast).
- `bash scripts/healthcheck.sh` (confirms every component is healthy and producing data).
- `bash scripts/fault_injection.sh full-scenario` (re-runs the Kafka and Citus fault-tolerance scenarios of Sections VI-B–C).
- `docker compose up -d --scale spark-worker=3` (re-runs the Spark scale-out of Section VI-D).
- Grafana at `http://localhost:3000` and the Spark master UI at `http://localhost:8080` provide the live views of Figs. 2 and 5.

E. Verification Queries for the Anomaly-Detection Fix

Stage 2 (rate-normalized, original IP pool) – which IPs were still flagged on a large batch:

```
SELECT ip_address, COUNT(*) FROM events_flagged
WHERE batch_id = <id> GROUP BY ip_address ORDER
BY COUNT(*) DESC;
```

Stage 3 (rate-normalized + widened IP pool) – final verification by event timestamp, independent of batch-ID numbering across container restarts:

```
SELECT COUNT(*) AS total_flagged, COUNT(DISTINCT
ip_address) AS distinct_flagged_ips FROM
events_flagged WHERE event_ts BETWEEN
‘<window_start>’ AND ‘<window_end>’;
```

On the verification run reported in Section VII-A and Fig. 6, this returned total_flagged = 26,946 and distinct_flagged_ips = 8—exactly the eight seeded bot addresses, with zero ordinary IPs included.